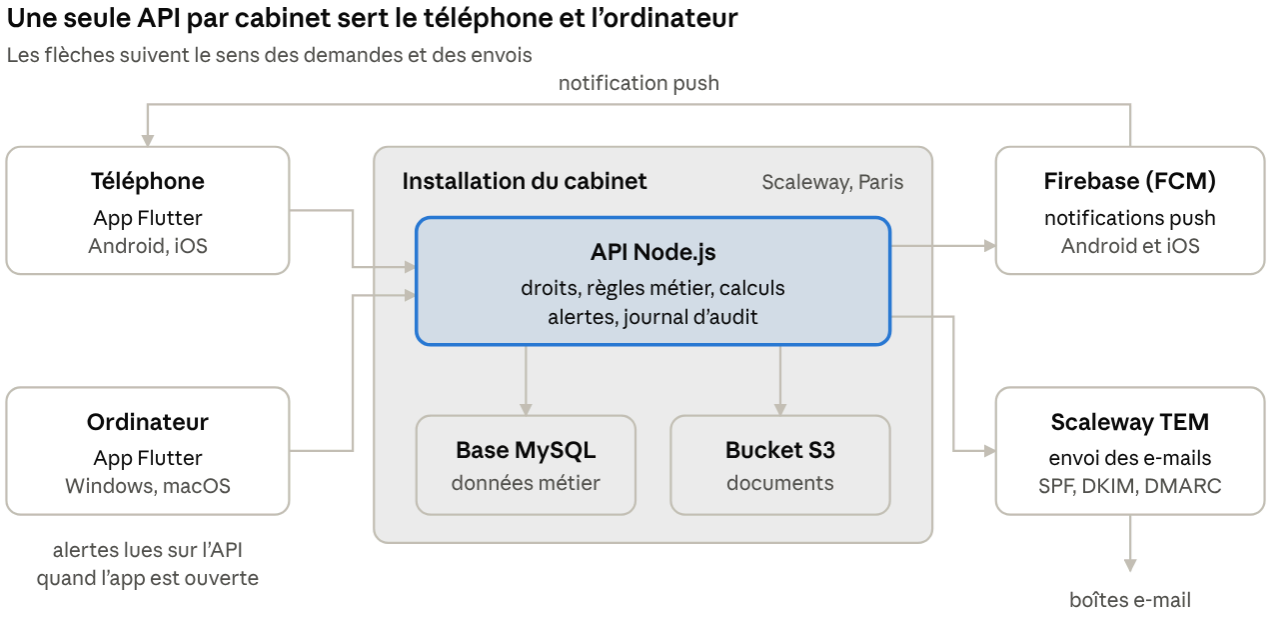
Application de gestion de cabinet de courtage — Cahier technique

Oct 21, 2026 · @Dorian

Ce document décrit la manière dont l'application est construite : architecture, technologies retenues et justifications, sécurité et déploiement.

Architecture générale



architecture · 2 applications, 1 installation par cabinet, 2 services externes

Les versions mobile et ordinateur communiquent avec la même API, en HTTPS. Elles partagent ainsi le même catalogue et les mêmes données. Seule l'API accède à la base de données, aux documents et aux services d'envoi.

Choix techniques et justifications

Les choix reposent sur trois principes : une seule base de code pour le mobile et l'ordinateur, des technologies déjà maîtrisées (Flutter, Node.js, SQL) et un hébergement des données en France.

Besoin	Choix	Justification
Application mobile et ordinateur	Flutter (Dart)	Une seule base de code pour Android, iOS, Windows et macOS. Technologie déjà utilisée sur un projet précédent.
Serveur (API)	Node.js, framework Express	Technologie maîtrisée. Express est minimaliste : routes, middlewares et gestion des erreurs sont écrits explicitement, ce qui facilite l'apprentissage ; c'est aussi le framework Node.js le plus documenté. Bibliothèques disponibles pour la base SQL, le stockage S3, les e-mails et les notifications.
Base de données	MySQL, en conteneur Docker	Données fortement liées (un client possède des contrats, un contrat génère des commissions). Les clés étrangères garantissent la cohérence.
Stockage des documents	Scaleway Object Storage, compatible S3	Volume impossible à estimer : la facturation suit l'usage réel (environ 0,016 € par Go et par mois à Paris). La compatibilité S3 permet de changer de fournisseur sans réécrire le code.
Envoi des e-mails	Scaleway Transactional Email (TEM)	Envoi depuis le serveur, avec le domaine de l'application comme expéditeur. Gestion de l'authentification du domaine (SPF, DKIM, DMARC) pour éviter le classement en spam. Offre de base : 300 e-mails inclus, puis 0,25 € les 1 000.

Besoin	Choix	Justification
Notifications sur téléphone	Firebase Cloud Messaging (FCM)	Service standard et gratuit de notifications push pour Android et iOS.
Alertes sur ordinateur	Notifications locales (flutter_local_notifications)	FCM ne prend pas en charge Windows. L'application ouverte récupère donc les alertes auprès du serveur et les affiche.
Hébergement	Scaleway, région Paris	Données hébergées en France (RGPD). Un seul fournisseur pour le serveur, le stockage et les e-mails.
Déploiement	Docker et Docker Compose	Installation identique pour chaque cabinet, créée en quelques commandes.

Application Flutter

Une application unique, compilée pour Android et iOS (téléphones) ainsi que pour Windows et macOS (ordinateurs).

Rôle de l'application

- Affichage des écrans, contrôle de la saisie (champs obligatoires, formats) et envoi des requêtes à l'API.
- Aucune règle déterminante côté application : les droits, les calculs de commission et le catalogue proviennent toujours du serveur.
- Seul le jeton de connexion est conservé sur l'appareil, dans le stockage sécurisé du système.

Cohérence entre mobile et ordinateur

- Chaque écran charge ses données depuis l'API à son ouverture. Une actualisation manuelle est disponible (glisser vers le bas sur mobile, bouton sur ordinateur).
- Toute modification est enregistrée sur le serveur et devient visible sur les autres appareils dès leur prochaine actualisation.

Organisation du code

- Écrans (une page par fonctionnalité), modèles (objets métier), services (appels à l'API), gestion d'état avec Provider.

- Mise en page adaptée à la taille de l'écran : une colonne sur téléphone, menu latéral sur ordinateur.

Gestion d'état

La gestion d'état désigne la manière dont l'application conserve les données affichées (utilisateur connecté, liste des clients, alertes) et met à jour les écrans lorsqu'elles changent. Exemple : après l'ajout d'un client, la liste se met à jour sans recharger l'écran.

- Solution retenue : **Provider**, utilisée dans la documentation officielle de Flutter pour introduire le sujet. Simple à comprendre et suffisante pour ce projet.
- Les données partagées entre plusieurs écrans (session, catalogue, alertes) sont placées dans des classes dédiées ; les écrans s'y abonnent et se redessinent automatiquement.

Contraintes

- La compilation et la publication des versions iOS et macOS nécessitent un Mac : elles seront réalisées avec l'aide d'une personne qui en dispose. La publication requiert aussi un compte développeur Apple (payant), de préférence au nom de l'éditeur de l'application.
- Pour la démonstration du BTS, les versions Android et Windows suffisent.

API Node.js

L'API est le seul composant qui accède à la base de données, aux documents et aux services d'envoi. C'est elle qui autorise ou refuse chaque requête.

Fonctionnement

- API REST : l'application envoie des requêtes HTTPS (GET, POST, PUT, DELETE) et reçoit des réponses au format JSON.
- Une ressource par domaine fonctionnel : authentification, catalogue, clients, contrats, commissions, documents, alertes, rendez-vous, paramétrage.

Authentification

- Connexion par e-mail et mot de passe. Le mot de passe est stocké haché (bcrypt ou argon2), jamais en clair.
- Après connexion, l'API délivre un jeton JWT de courte durée, transmis avec chaque requête, ainsi qu'un jeton de renouvellement évitant de ressaisir le mot de passe.
- Le nombre de tentatives de connexion est limité afin de bloquer les attaques par force brute.

Contrôle des droits (admin / user)

Chaque requête fait l'objet de trois vérifications :

1. Validité du jeton. À défaut : refus (401).
2. Autorisation du rôle. Exemple : seul l'admin peut modifier le catalogue. À défaut : refus (403).
3. Propriété de la donnée. Exemple : un user n'accède qu'à ses propres clients et aux produits qui lui sont ouverts. À défaut : refus (403 ou 404).

Traitements automatiques

- Quotidiennement : calcul des alertes (pièces manquantes, contrats bloqués, échéances, commissions non reçues), puis envoi des notifications et des e-mails.
- Hebdomadairement : envoi de la synthèse à l'admin.
- Le journal d'audit est alimenté directement par l'API, afin qu'aucune action ne puisse y échapper.

Base de données

Une base MySQL par cabinet, exécutée dans un conteneur Docker et contenant l'ensemble des données métier. Les fichiers sont stockés sur S3 ; la base n'en conserve que les métadonnées.

Règles de conception

- Clés étrangères sur toutes les relations : un contrat ne peut pas exister sans client.
- Montants (primes, taux, commissions) en type `DECIMAL`, jamais en `FLOAT`, afin d'éviter les erreurs d'arrondi.
- Dates de création et de modification sur chaque table ; archivage plutôt que suppression lorsque l'historique doit être conservé.
- Requêtes préparées exclusivement : aucune saisie utilisateur n'est insérée directement dans une requête SQL.
- Évolutions de la structure gérées par des scripts de migration, exécutés de manière identique sur chaque installation.

Travaux à réaliser

- [ ] MCD : entités, attributs, associations et cardinalités, à partir du cahier des charges.
- [ ] MLD : tables, clés primaires et clés étrangères.
- [ ] Script SQL de création de la base.
- [ ] Jeu de données fictives pour les tests et la démonstration.

Stockage des documents (S3)

Les pièces (PDF, images) sont stockées dans Scaleway Object Storage, compatible S3, région Paris. La capacité s'adapte automatiquement au volume, sans dimensionnement préalable.

- **Un bucket privé par cabinet** : aucun fichier n'est accessible publiquement.
- **Nom de fichier généré** (identifiant unique) à la place du nom d'origine, pour éviter les collisions.
- **Métadonnées en base** : nom d'origine, type, taille, date, auteur, client ou contrat associé, emplacement dans le bucket.
- **Dépôt** : l'application transmet le fichier à l'API, qui contrôle les droits, le type (PDF, JPEG, PNG) et la taille maximale (10 Mo par fichier) avant de l'enregistrer.
- **Téléchargement** : après contrôle des droits, l'API génère un lien temporaire (URL présignée) valable quelques minutes.
- **Coût** : environ 0,016 € par Go et par mois à Paris ; 75 Go de trafic sortant inclus chaque mois.

E-mails, notifications et alertes

Les alertes sont générées par le serveur et enregistrées en base. Le téléphone, l'ordinateur et les e-mails ne sont que des canaux de diffusion : une alerte lue sur un appareil l'est donc sur tous.

Canal	Disponibilité	Fonctionnement
Notification push (téléphone)	Même application fermée	À la connexion, l'application transmet à l'API l'identifiant de l'appareil (jeton FCM). L'API envoie la notification à Firebase, qui la remet au téléphone (via le service d'Apple pour iOS).
Alerte (ordinateur)	Application ouverte	L'application interroge l'API toutes les 5 minutes, ainsi qu'à l'ouverture et à chaque actualisation, met à jour le compteur d'alertes et affiche une notification Windows ou macOS.
E-mail	Relances, rappels, synthèse	L'API envoie les messages via Scaleway TEM, depuis le domaine de l'application. Ce domaine est authentifié (SPF, DKIM, DMARC) pour éviter le classement en spam.

À terme, l'interrogation régulière sur ordinateur pourra être remplacée par une connexion en temps réel (WebSocket).

Sécurité et RGPD

La sécurité repose sur le serveur : l'application pouvant être copiée ou imitée, tout contrôle effectué uniquement côté application peut être contourné.

Risque	Mesure
Vol des mots de passe	Hachage (bcrypt ou argon2), aucun stockage en clair
Attaque par force brute	Limitation du nombre de tentatives de connexion
Interception des échanges	HTTPS obligatoire entre l'application et l'API
Accès aux données d'un autre user	Contrôle du rôle et de la propriété des données à chaque requête
Injection SQL	Requêtes préparées exclusivement
Données invalides ou malveillantes	Validation de chaque entrée côté serveur (types, longueurs, formats)

Risque	Mesure
Fuite de documents	Bucket privé, liens de téléchargement temporaires
Fuite des clés secrètes (S3, e-mails, Firebase)	Variables d'environnement, jamais dans le code ni sur GitHub
Contestation d'une action	Journal d'audit (auteur, action, date)

RGPD

- Collecte limitée aux données nécessaires à la gestion des contrats.
- Export et suppression des données d'un client sur demande.
- Conservation des données jusqu'à une demande de suppression ; la suppression est alors définitive, documents stockés sur S3 compris.
- Registre des traitements tenu par chaque cabinet ; contrat de sous-traitance conclu entre nous, en tant qu'hébergeur, et chaque cabinet.
- Utilisation exclusive de données fictives pour le développement et la démonstration.

Hébergement, déploiement et sauvegardes

L'ensemble est hébergé chez Scaleway, région Paris. Chaque cabinet dispose d'une installation dédiée, isolée des autres.

Composition d'une installation

- Un conteneur Docker pour l'API Node.js.
- Un conteneur Docker pour la base MySQL. Les sauvegardes sont donc à notre charge (voir ci-dessous).
- Un bucket S3 privé pour les documents.
- Un chemin dédié sur le domaine de l'application (ex. : nomdelapplication.fr/nomducabinet), en HTTPS. Un reverse proxy unique, avec un certificat Let's Encrypt, redirige chaque chemin vers l'API du cabinet concerné. Au premier lancement, l'utilisateur saisit l'identifiant de son cabinet ; l'application en déduit l'adresse de son serveur.

Déploiement

- Un fichier Docker Compose décrit l'installation ; un nouveau cabinet correspond à une copie avec ses propres paramètres (variables d'environnement).
- Mise à jour : publication d'une nouvelle version de l'image Docker, déployée cabinet par cabinet ; les migrations de base s'exécutent au démarrage.

- Trois environnements : développement (poste local), démonstration (données fictives), production (un par cabinet client).

Sauvegardes

- Sauvegarde automatique quotidienne de chaque base, stockée dans un bucket S3 distinct.
- Conservation des sauvegardes pendant 30 jours : chaque jour, la plus ancienne est supprimée.
- Tests de restauration réguliers, seul moyen de garantir que les sauvegardes sont exploitables.

Outils, tests et point à définir

Outils

- Git et GitHub : versionnage du code, une branche par fonctionnalité.
- VS Code pour l'API et Flutter ; Android Studio pour l'émulateur Android.
- Postman ou Bruno pour tester l'API indépendamment de l'application.
- Trello pour la planification et le suivi des tâches.

Tests

- Tests unitaires sur les règles critiques : calcul et répartition des commissions, rapprochement attendu / reçu, contrôle des droits.
- Tests de chaque route de l'API (cas autorisés et cas refusés).
- Tests des principaux écrans Flutter.
- Cahier de recette déroulé avec le client avant la mise en service.

Point à définir ultérieurement

- [ ] Compte développeur Apple : titulaire du compte utilisé pour publier la version iOS.

Sources

- Scaleway : tarifs du stockage (Object Storage)
- Scaleway : Transactional Email
- firebase_messaging (plateformes prises en charge)
- flutter_local_notifications (plateformes prises en charge)